

iOS 项目中 JSON 的解析与转换

iOS 项目 JSON 解析机制详解

项目采用“JSONKit 底层解析 + Mantle 模型映射”的组合方案，将服务器返回的 JSON 数据转换为代码可直接使用的 Model 对象，实现“类型安全、结构清晰、易于维护”的解析效果。

一、JSON 解析的基本流程

整体流程可概括为“字符串→字典→模型”的两步转换，核心依赖 JSONKit 与 Mantle 两个工具：

步骤	工具 / 组件	核心作用	输出结果
1. 接收数据	网络请求框架 (MKNetworkModel)	从服务器获取原始 JSON 字符串	JSON 格式字符串
2. 字符串转字典	JSONKit	底层解析 JSON 字符串，生成 NSDictionary/NSArray	字典 / 数组（未类型化）
3. 字典转模型	Mantle (MTLJSONAdapter)	按规则将字典映射为自定义 Model 对象	强类型 Model 对象

大白话解释：JSONKit 负责“读懂”JSON 字符串并转为通用字典，Mantle 负责“翻译”字典为业务层可直接使用的 Model（如 UserModel、NewsModel），避免手动解析的繁琐与错误。

二、Mantle (MTLJSONAdapter) 的使用方式

Mantle 是核心模型映射工具，通过定义“JSON 字段与 Model 属性的对应关系”，自动完成数据转换，支持简单类型、枚举、数组、嵌套对象等复杂场景。

1. 基本映射（JSON 字段与 Model 属性对应）

在自定义 Model 类中实现 +JSONKeyPathsByPropertyKey 方法，定义 JSON 字段与 Model 属性的映射规则（字段名一致可省略，不一致需显式指定）。

示例代码（UserModels.m）：

```
#import <Mantle/Mantle.h>
@interface UserModels : MTLModel <MTLJSONSerializing>
@property (nonatomic, assign) NSInteger code;    // 状态码（JSON 字段：code）
@property (nonatomic, copy) NSString *msg;       // 提示信息（JSON 字段：msg）
@property (nonatomic, strong) NSArray *items;    // 用户列表（JSON 字段：users，字段名不一致）
@end
@implementation UserModels
// 定义 JSON 字段与 Model 属性的映射关系
+ (NSDictionary *)JSONKeyPathsByPropertyKey {
    return @{
        @"code": @"code",    // JSON 字段与属性名一致，直接映射
        @"msg": @"msg",      // JSON 字段与属性名一致，直接映射
        @"items": @"users"   // JSON 字段“users”映射到 Model 属性“items”
    };
}
@end
```

2. 类型转换（处理数据类型不匹配）

当 JSON 中的数据类型与 Model 属性类型不一致时（如 JSON 是数字、Model 是字符串），需通过 MTLValueTransformer 定义转换规则。

（1）简单类型转换（数字 字符串）

示例代码（UserModels.m）：

```
// Model 属性：itemId（字符串类型），JSON 字段：id（数字类型）
@property (nonatomic, copy) NSString *itemId;
// 定义“数字→字符串”“字符串→数字”的双向转换规则
+ (NSValueTransformer *)itemIdJSONTransformer {
    // reversibleTransformerWithForwardBlock：正向转换（JSON→Model）
    // reverseBlock：反向转换（Model→JSON，如需要上传 Model 时）
    return [MTLValueTransformer reversibleTransformerWithForwardBlock:^(NSNumber *num)
```

```

{
    // 正向：JSON 数字 → Model 字符串
    return [num stringValue];
} reverseBlock:^(NSString *numStr) {
    // 反向：Model 字符串 → JSON 数字
    return [NSNumber numberWithUnsignedLongLong:[numStr longLongValue]];
}];
}

```

(2) 枚举类型转换（数字 枚举）

当 JSON 用数字表示状态 / 类型时，可映射为代码中的枚举，提升可读性。

示例代码（HotNewsDetailsModel.m）：

```

// 定义新闻类型枚举
typedef NS_ENUM(NSInteger, HotNewsType) {
    HotNewsTypeText = 0, // 文本新闻
    HotNewsTypeImage = 1 // 图片新闻
};

@interface HotNewsDetailsModel : MTLModel <MTLJSONSerializing>
@property (nonatomic, assign) HotNewsType type; // 新闻类型（JSON 字段：type，数字）
@end

@implementation HotNewsDetailsModel
// 定义 JSON 数字 → 枚举的转换规则
+ (NSValueTransformer *)typeJSONTransformer {
    return [NSValueTransformer mtl_valueMappingTransformerWithDictionary:@{
        @(0): @(HotNewsTypeText), // JSON 数字 0 → 枚举 HotNewsTypeText
        @(1): @(HotNewsTypeImage) // JSON 数字 1 → 枚举 HotNewsTypeImage
    }];
}
@end

```

3. 数组转换（JSON 数组 → Model 数组）

当 JSON 字段是数组（如 `users: [{}, {}]`）时，需定义“数组元素→单个 Model”的转换规则，生成强类型的 Model 数组。

示例代码（TagsModel.m）：

```

#import <Mantle/Mantle.h>
#import "TagModel.h" // 数组元素对应的 Model 类
@interface TagsModel : MTLModel <MTLJSONSerializing>
@property (nonatomic, strong) NSArray<TagModel *> *items; // TagModel 数组 (JSON 字段 : tags)
@end

@implementation TagsModel
// 定义 JSON 数组 → Model 数组的转换规则
+ (NSValueTransformer *)itemsJSONTransformer {
    return [MTLValueTransformer transformerWithBlock:^(NSArray *jsonArray) {
        NSMutableArray *modelArray = [NSMutableArray array];

        // 遍历 JSON 数组，逐个转换为 TagModel
        [jsonArray enumerateObjectsUsingBlock:^(id obj, NSUInteger idx, BOOL *stop) {
            NSDictionary *itemDict = (NSDictionary *)obj; // 单个 JSON 元素 (字典)
            NSError *error = nil;

            // 将字典转换为 TagModel
            TagModel *tagModel = [MTLJSONAdapter modelOfClass:[TagModel class]
                               fromJSONDictionary:itemDict
                               error:&error];

            if (error) {
                NSLog(@"TagModel 转换失败: %@", error.localizedDescription);
            } else if (tagModel) {
                [modelArray addObject:tagModel];
            }
        }];

        return modelArray; // 返回转换后的 Model 数组
    }];
}

// 映射 JSON 字段 “tags” 到 Model 属性 “items”
+ (NSDictionary *)JSONKeyPathsByPropertyKey {
    return @{@"items": @"tags"};
}

@end

```

4. 嵌套对象转换 (处理复杂 JSON 结构)

当 JSON 存在嵌套结构（如 `user: { "info": { "name": "张三" } }`）时，需指定嵌套对象对应的 Model 类，自动完成多层转换。

示例代码（HotNewDetailsModel.m）：

```
#import <Mantle/Mantle.h>
#import "NewsInfoModel.h" // 嵌套对象对应的 Model 类
@interface HotNewDetailsModel : MTLModel <MTLJSONSerializing>
@property (nonatomic, strong) NewsInfoModel *mod; // 嵌套的新闻详情（JSON 字段：info）
@end
@implementation HotNewDetailsModel
// 定义嵌套对象的转换规则（直接指定 Model 类）
+ (NSValueTransformer *)modJSONTransformer {
    // mtl_JSONDictionaryTransformerWithModelClass：自动将 JSON 字典转换为指定 Model
    return [NSValueTransformer mtl_JSONDictionaryTransformerWithModelClass:[
NewsInfoModel class]];
}
// 映射 JSON 字段 “info” 到 Model 属性 “mod”
+ (NSDictionary *)JSONKeyPathsByPropertyKey {
    return @{@"mod": @"info"};
}
@end
```

三、底层解析（JSONKit 的作用）

JSONKit 是轻量级高性能的 JSON 解析库，负责最底层的“JSON 字符串→字典 / 数组”转换，为 Mantle 提供原始数据。其核心逻辑是遍历 JSON 字符串，按语法规则解析为 Objective-C 对象（-NSDictionary、NSArray、NSString、NSNumber 等）。

核心代码片段（JSONKit.m）：

```
// JSONKit 底层解析函数：将 JSON 字符串解析为对象
static id json_parse_it(JKParseState *parseState) {
    id parsedObject = NULL;
    int stopParsing = 0;

    // 遍历 JSON 字符串，按字符逐个解析（处理 {}、[]、字符串、数字等语法）
    while ((JK_EXPECT_T(stopParsing == 0)) && (JK_EXPECT_T(parseState->atIndex < parseState
->stringBuffer.bytes.length))) {
```

```
unichar c = [parseState->stringBuffer.bytes characterAtIndex:parseState->atIndex];
```

```
switch (c) {
```

```
    case '{': // 解析对象 (JSON Object → NSDictionary)
```

```
        parsedObject = json_parse_object(parseState);
```

```
        stopParsing = 1;
```

```
        break;
```

```
    case '[': // 解析数组 (JSON Array → NSArray)
```

```
        parsedObject = json_parse_array(parseState);
```

```
        stopParsing = 1;
```

```
        break;
```

```
    case '\"': // 解析字符串 (JSON String → NSString)
```

```
        parsedObject = json_parse_string(parseState);
```

```
        stopParsing = 1;
```

```
        break;
```

```
    case 't': // 解析 true → YES (NSNumber)
```

```
    case 'f': // 解析 false → NO (NSNumber)
```

```
        parsedObject = json_parse_boolean(parseState);
```

```
        stopParsing = 1;
```

```
        break;
```

```
    case 'n': // 解析 null → nil
```

```
        parsedObject = json_parse_null(parseState);
```

```
        stopParsing = 1;
```

```
        break;
```

```
    case '0': case '1': case '2': case '3': case '4': // 解析数字 → NSNumber
```

```
    case '5': case '6': case '7': case '8': case '9':
```

```
    case '-':
```

```
        parsedObject = json_parse_number(parseState);
```

```
        stopParsing = 1;
```

```
        break;
```

```
    default: // 跳过空格、换行等无关字符
```

```
        parseState->atIndex++;
```

```
        break;
```

```
    }
```

```
}
```

```
return parsedObject; // 返回解析后的对象 (NSDictionary/NSArray/NSString/NSNumber/nil
```

```
)
```

```
}
```

四、在网络请求中的集成（MKNetworkModel）

项目将 JSON 解析与网络请求深度集成，在 `MKNetworkModel` 中封装“请求→解析→返回 Model”的完整流程，业务层无需手动处理解析逻辑。

示例代码（`MKNetworkModel.m`）：

```
// 请求并直接返回 Model 对象
- (MKNetworkOperationEx *)requestModelReturnWithParameter:(NSDictionary *)dic
    onKeyPath:(NSString *)keyPath
    withClass:(Class)aClass
    onSuccessed:(ResponseBlock)succeededBlock
    onError:(ErrorBlock)errorBlock {
    // 1. 显示加载指示器（MBProgressHUD）
    // ...（省略加载指示器逻辑）...
    // 2. 定义请求成功回调
    DEFINE_WEAK(self);
    DictionaryBlock OnSucceededBlock = ^(NSMutableDictionary *results) {
        // 隐藏加载指示器
        // ...（省略隐藏逻辑）...
        // 3. 按 keyPath 提取目标数据（如从 {"data": { ... }} 中提取 "data" 字段）
        if (keyPath && keyPath.length > 0) {
            results = results[keyPath];
        }
        // 4. 字典转 Model（核心解析步骤）
        NSError *error = nil;
        MTLModel *model = [MTLJSONAdapter modelOfClass:aClass
            fromJSONDictionary:results
            error:&error];
        // 5. 回调结果（成功返回 Model，失败返回错误）
        if (error) {
            if (errorBlock) errorBlock(error);
        } else {
            if (succeededBlock) succeededBlock(model, nil);
        }
    };
    // 6. 定义请求失败回调
    ErrorBlock OnErrorBlock = ^(NSError *error) {
        // ...（省略错误处理逻辑）...
        if (errorBlock) errorBlock(error);
    };
}
```

```
};
// 7. 发起网络请求（底层使用 MKNetworkEngineEx）
MKNetworkOperationEx *operation = [_netWorkEngineEx requestDictionaryRet-
urnWithParameter:dic
                                onKeyPath:keyPath
                                onSucceeded:OnSucceededBlock
                                onError:OnErrorBlock
                                WithHttpMethod:@"POST"
                                withSSL:NO];

_operation = operation;
return operation;
}
```

五、采用该解析方式的优势

1. **类型安全**：Model 属性为强类型（如 `NSInteger code`、`NSString *name`），编译时可检查类型错误，避免运行时因字典 key 错误或类型不匹配导致的崩溃；
2. **使用便捷**：通过点语法直接访问数据（如 `user.name`），替代繁琐的 `[dict objectForKey:@"name"]`，代码更简洁；
3. **结构清晰**：Model 类明确定义数据结构，业务逻辑与数据解析分离，便于维护与迭代；
4. **扩展性强**：支持复杂场景（枚举、数组、嵌套对象），新增字段仅需在 Model 中添加属性并配置映射；
5. **性能优异**：底层依赖 JSONKit，解析速度快、内存占用低，满足高并发场景需求。

六、实际解析示例

1. 服务器返回的 JSON 数据

```
{
  "code": 200,
  "msg": "success",
  "users": [
    {
      "id": 12345,
      "name": "张三",
    }
  ]
}
```



```
"jobs": [
  {"id": 1, "name": "工程师"}
]
}
```

2. 对应的 Model 结构

- **UserListModel**（根 Model）：包含 `code`（状态码）、`msg`（提示）、`items`（用户数组，映射 JSON 字段 `users`）；
- **UserModel**（用户 Model）：包含 `itemId`（用户 ID，JSON 字段 `id`，数字转字符串）、`name`（用户名）、`jobs`（职业数组，TagModel 类型）；
- **TagModel**（职业 Model）：包含 `id`（职业 ID）、`name`（职业名称）。

3. 解析结果

通过 `MKNetworkModel` 请求后，直接得到 `UserListModel` 对象：

```
// 业务层调用（无需手动解析）
UserListModel *userListModel = (UserListModel *)responseModel;
// 访问解析后的数据
NSLog(@"状态码： %ld", userListModel.code); // 输出： 200
NSLog(@"提示信息： %@", userListModel.msg); // 输出： success
// 访问用户数组
UserModel *firstUser = userListModel.items.firstObject;
NSLog(@"用户名： %@", firstUser.name); // 输出： 张三
NSLog(@"用户 ID： %@", firstUser.itemId); // 输出： 12345（字符串类型）
// 访问用户的职业
TagModel *job = firstUser.jobs.firstObject;
NSLog(@"职业名称： %@", job.name); // 输出： 工程师
```

（注：文档部分内容可能由 AI 生成）